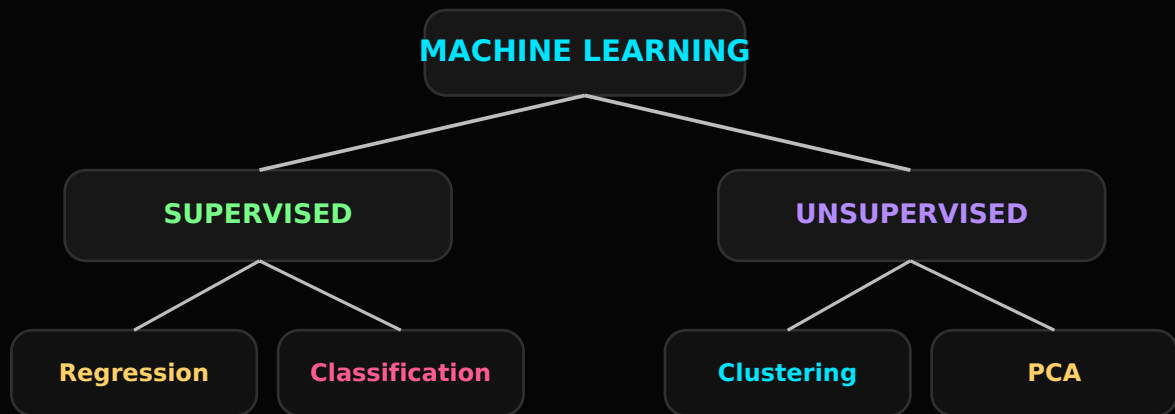


DATA ANALYTICS USING AI

MACHINE LEARNING

Interview Notes + Scikit-learn + Practical Cheat Sheet



Designed for: Data Analyst / Data Engineer Trainee / Fresher interviews

Learning goal: Explain the concept simply, choose the correct model, use Scikit-learn, and evaluate results.

Made By SK SAHIL

(Freelance AI Engineer)

1. Interview Questions - Data Analytics Role

Answer in 20-40 seconds. Keep the definition + one example.

Q1. What is Machine Learning?

Answer: ML allows a system to learn patterns from historical data and make predictions or decisions without writing a separate rule for every case.

Q2. Supervised vs unsupervised learning?

Supervised: target/label exists (price, churn). Unsupervised: no target; discover structure (customer groups).

Q3. Regression vs classification?

Regression predicts a number. Classification predicts a class/category.

Q4. What are features and target?

Features (X) are input columns. Target (y) is the value we want to predict.

Q5. Why split data into train and test sets?

Train data teaches the model; test data checks performance on unseen records. It estimates generalization.

Q6. What is overfitting?

The model memorizes training data and performs poorly on new data. Reduce complexity, add data, regularize, or use cross-validation.

Q7. What is underfitting?

The model is too simple and performs poorly on both training and test data.

Q8. Why is data preprocessing important?

Models need clean, numeric, consistent inputs. We handle missing values, categories, scales, outliers, and duplicates.

Q9. What is feature scaling?

It puts numeric features on comparable scales. Common tools: StandardScaler and MinMaxScaler.

Q10. Accuracy vs precision vs recall?

Accuracy: overall correct. Precision: predicted positives that were correct. Recall: actual positives successfully found.

Q11. When is accuracy misleading?

With imbalanced classes. Example: 99% non-fraud can give 99% accuracy even if the model misses all fraud.

Q12. What is a confusion matrix?

A table of TP, TN, FP, FN used to understand classification errors.

2. Interview Questions - Data Engineer Trainee

For trainee roles, interviewers usually expect ML awareness plus strong data pipeline thinking.

Q1. What does a data engineer do in an ML project?

Builds reliable pipelines to collect, clean, transform, store, and deliver data for analytics or model training.

Q2. ETL vs ELT?

ETL: Extract -> Transform -> Load. ELT: Extract -> Load -> Transform, often used with modern cloud warehouses.

Q3. Structured vs unstructured data?

Structured: rows/columns. Unstructured: text, images, audio, documents.

Q4. Batch vs streaming data?

Batch: process groups periodically. Streaming: process events continuously/near real time.

Q5. Why is data quality important for ML?

Bad or inconsistent data creates unreliable features and unreliable predictions. Common checks: nulls, duplicates, ranges, types, freshness.

Q6. What is data leakage?

Information from the future/test set accidentally enters training. It makes evaluation unrealistically high.

Q7. Why use a Pipeline in Scikit-learn?

It chains preprocessing and modeling consistently, reduces leakage risk, and makes training/evaluation repeatable.

Q8. Why should preprocessing be fitted only on training data?

Because using test-set statistics leaks information from the test set into the model-building process.

Q9. What is feature engineering?

Creating useful model inputs from raw data, such as age from DOB, day-of-week from timestamp, or total spend from transactions.

Q10. What is model drift?

The relationship between incoming data and the target changes over time, so model performance can decline.

Q11. What should be stored for reproducible ML?

Dataset/version, feature logic, code, model parameters, evaluation metrics, random seed, and trained model artifact.

Q12. SQL or ML - which is more important for a Data Engineer trainee?

SQL and data engineering fundamentals first. ML knowledge helps you understand how downstream models consume data.

3. Machine Learning - Simple Notes to Memorize

Think: problem -> data -> pattern -> prediction -> evaluation.



Memorize this sequence for interviews and practical work.

AI

Broad field: machines perform tasks that normally need human intelligence.

Machine Learning

Subset of AI: learns patterns from data.

Deep Learning

Subset of ML using multi-layer neural networks. Not required for a basic Data Analytics course.

Dataset

Collection of rows (records) and columns (features).

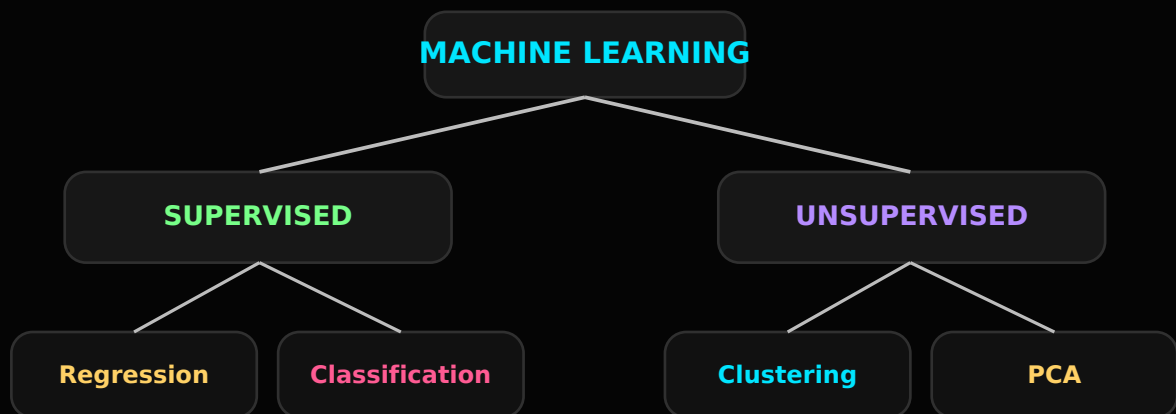
Feature (X)

Input variable used to make a prediction. Example: age, income, clicks.

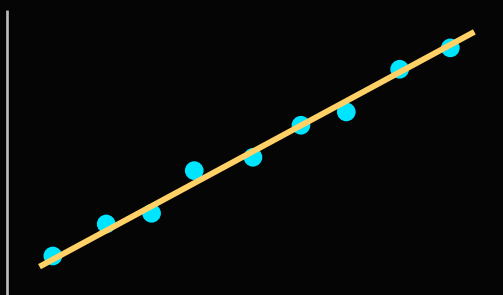
Target (y)

Output to predict. Example: price, churn, fraud.

Core Types

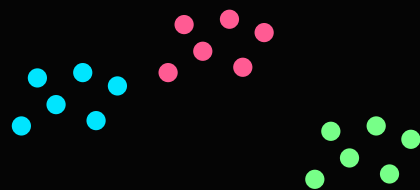


Regression: predict a number



Example: sales, salary, price

Clustering: group similar records



Example: customer segmentation

4. Algorithms You Actually Need

Task	Algorithm	Remember it like this	Typical use
Regression	Linear Regression	Fits a best-fit relationship	Sales / price / salary
Regression	Decision Tree Regressor	Rules split the data	Non-linear tabular data
Regression	Random Forest Regressor	Many trees vote/average	Strong tabular baseline
Classification	Logistic Regression	Probability for class	Churn / yes-no
Classification	KNN	Nearby points decide	Small/simple datasets
Classification	Decision Tree	If-else style rules	Easy interpretation
Classification	Random Forest	Many trees vote	Strong general baseline
Classification	Naive Bayes	Probability + independence assumption	Text/spam basics
Classification	SVM	Finds a separating boundary	Smaller scaled datasets
Clustering	K-Means	Groups points around centroids	Customer segmentation
Dimension reduction	PCA	Compresses features while preserving variation	Visualization / simplification

Rule of thumb

- Predict a number? Start with Linear Regression; compare tree-based models.
- Predict a category? Start with Logistic Regression; compare Random Forest / other models.
- No target? Try clustering; K-Means is the essential beginner algorithm.

5. Data Preprocessing - Before Training

Missing values

Use drop only when safe; otherwise impute. SimpleImputer: mean/median/most_frequent.

Duplicates

Remove repeated records when they are truly duplicates.

Categorical data

Convert text categories to numeric representation. OneHotEncoder is the standard for nominal categories.

Feature scaling

StandardScaler: centers/scales. MinMaxScaler: maps values to a range (commonly 0-1).

Outliers

Detect with domain rules, IQR, boxplots, z-scores. Do not delete automatically.

Feature selection

Remove useless/redundant columns; keep variables relevant to the target.

Train-test split

Typical beginner split: 80/20. Use random_state for reproducibility.

Stratification

For classification, preserve class proportion with stratify=y when appropriate.

Critical interview rule

Never learn preprocessing from the test set

Fit scalers, imputers, encoders and feature selection on training data only. Pipelines help prevent leakage.

6. Model Evaluation - Know These Metrics

Regression Metrics

Metric	Meaning	Best direction
MAE	Average absolute error. Easy to explain.	Lower
MSE	Squares errors; penalizes large mistakes more.	Lower
RMSE	Square root of MSE; same unit as target.	Lower
R ²	How much variation is explained by the model.	Higher (usually)

Classification Metrics

ACTUAL	Positive	TP Correct +	FN Missed +
	Negative	FP False alarm	TN Correct -

Accuracy

Correct predictions / all predictions.

Precision

Of predicted positives, how many were truly positive?

Recall

Of actual positives, how many did we find?

F1-score

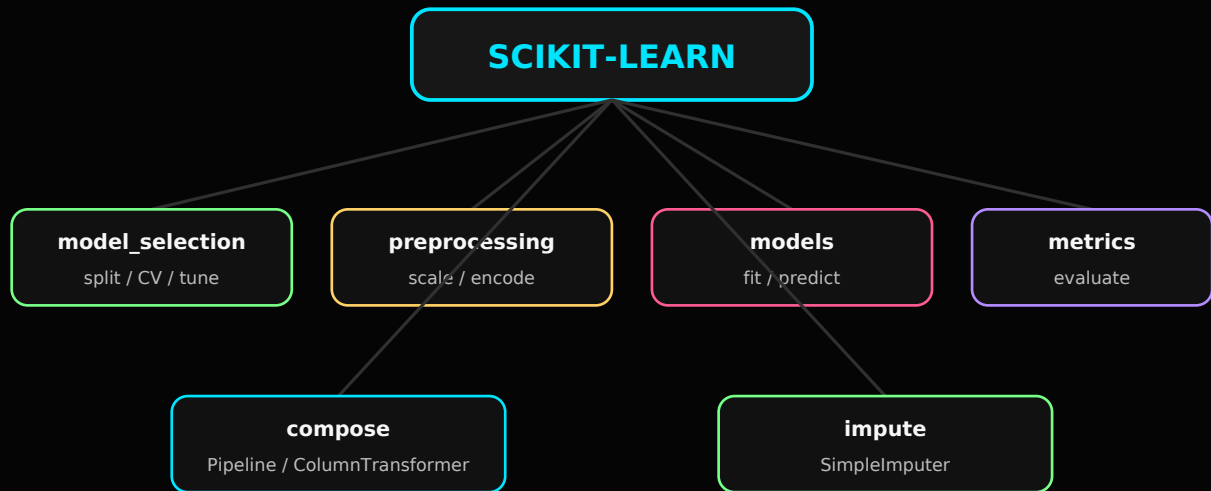
Balance of precision and recall.

ROC-AUC

Measures ranking/separation across classification thresholds.

7. Scikit-learn - What Students Must Memorize

For this Data Analytics course, use Scikit-learn for most practical ML. Learn the workflow and core APIs - not every class.



Module	Memorize
<code>sklearn.model_selection</code>	<code>train_test_split</code> , <code>cross_val_score</code> , <code>GridSearchCV</code>
<code>sklearn.preprocessing</code>	<code>StandardScaler</code> , <code>MinMaxScaler</code> , <code>OneHotEncoder</code> , <code>LabelEncoder</code> *
<code>sklearn.impute</code>	<code>SimpleImputer</code>
<code>sklearn.linear_model</code>	<code>LinearRegression</code> , <code>LogisticRegression</code>
<code>sklearn.tree</code>	<code>DecisionTreeRegressor</code> , <code>DecisionTreeClassifier</code>
<code>sklearn.ensemble</code>	<code>RandomForestRegressor</code> , <code>RandomForestClassifier</code>
<code>sklearn.neighbors</code>	<code>KNeighborsClassifier</code>
<code>sklearn.naive_bayes</code>	<code>GaussianNB</code> / <code>MultinomialNB</code>
<code>sklearn.svm</code>	<code>SVC</code> / <code>SVR</code> (basic awareness)
<code>sklearn.cluster</code>	<code>KMeans</code>
<code>sklearn.decomposition</code>	<code>PCA</code>
<code>sklearn.metrics</code>	<code>MAE</code> , <code>MSE/RMSE</code> , R^2 , accuracy, precision, recall, <code>F1</code> , confusion matrix, <code>ROC-AUC</code>
<code>sklearn.pipeline</code>	<code>Pipeline</code> / <code>make_pipeline</code>
<code>sklearn.compose</code>	<code>ColumnTransformer</code>

*`LabelEncoder` is mainly for target labels; use `OneHotEncoder`/`ColumnTransformer` for input categories in typical tabular workflows.

8. Scikit-learn Mental Model: fit -> transform -> predict

Method	Meaning	Used on
fit(X, y)	Learn parameters/patterns from training data	Models + transformers
transform(X)	Apply learned preprocessing	Scaler / encoder / imputer
fit_transform(X)	Fit then transform training data	Transformers
predict(X)	Generate predicted class/value	Trained model
predict_proba(X)	Class probabilities (if supported)	Classifiers
score(X, y)	Estimator default score	Quick check; not always enough

Minimal Regression Pattern

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(mean_absolute_error(y_test, pred), r2_score(y_test, pred))
```

Minimal Classification Pattern

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(accuracy_score(y_test, pred))
print(classification_report(y_test, pred))
```

9. Pipelines, Cross-Validation & Tuning

Pipeline

Chains preprocessing + model into one repeatable object. Helps avoid inconsistent preprocessing and leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression())
])

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)
```

ColumnTransformer

Apply different preprocessing to different columns: e.g., scale numeric columns and one-hot encode categorical columns.

Cross-validation

Train/evaluate across multiple folds to get a more reliable estimate than one split.

GridSearchCV

Tests combinations of hyperparameters using cross-validation and selects the best configuration according to a chosen score.

Memorize the difference

Term	Meaning
Parameter	Learned by the model from data. Example: regression coefficients.
Hyperparameter	Chosen before/during training. Example: n_estimators, max_depth, C, k.
Model selection	Compare algorithms or hyperparameter settings.
Cross-validation	Repeated train/validation splits for more reliable evaluation.

10. What to Practice in a Data Analytics Course

Recommended Mini Projects

Sales Prediction - Regression

Clean sales data -> EDA -> Linear/Random Forest -> MAE/RMSE/R².

Customer Churn - Classification

Encode customer data -> Logistic/Random Forest -> precision/recall/F1.

Customer Segmentation - Clustering

Scale numeric behavior features -> K-Means -> interpret clusters.

Fraud / Risk Screening - Classification

Handle imbalance carefully -> confusion matrix -> recall/precision.

Libraries - How Much?

Library	Course depth	Why
Pandas	HIGH	Cleaning, filtering, joins, grouping, feature creation
NumPy	BASIC-MEDIUM	Arrays, numerical operations
Matplotlib	MEDIUM	Core plotting
Seaborn	MEDIUM	Fast statistical/EDA plots
Scikit-learn	HIGH	Main ML library for this course
XGBoost	INTRO ONLY	Optional strong boosting model for tabular data
Statsmodels	OPTIONAL	Statistical inference / classical regression detail
TensorFlow / PyTorch	NOT REQUIRED	Deep learning is outside basic Data Analytics ML

Suggested balance

70% analytics foundations (Excel/SQL/Python/Pandas/EDA) + 25% practical ML with Scikit-learn + 5% AI-assisted analytics workflow.

11. FINAL MACHINE LEARNING CHEAT SHEET

One-page revision before interview / test

CORE

- ML = learn patterns from data.
- X = features; y = target.
- Supervised = target exists; unsupervised = no target.
- Regression = number; classification = category; clustering = groups.

WORKFLOW

- Problem -> Data -> Clean/EDA -> X/y -> Split -> Train -> Predict -> Evaluate -> Improve.
- Never fit preprocessing on the test set.

PREPROCESS

- Missing -> SimpleImputer.
- Categories -> OneHotEncoder.
- Scale -> StandardScaler / MinMaxScaler.
- Split -> train_test_split(..., test_size=0.2, random_state=42).

ALGORITHMS

- Regression: Linear, Decision Tree, Random Forest.
- Classification: Logistic, KNN, Decision Tree, Random Forest, Naive Bayes, SVM.
- Clustering: K-Means. Reduction: PCA.

METRICS

- Regression: MAE, MSE, RMSE lower; R^2 higher.
- Classification: Accuracy, Precision, Recall, F1, Confusion Matrix, ROC-AUC.
- Imbalanced classes -> do not trust accuracy alone.

SCIKIT-LEARN

- fit = learn; transform = preprocess; predict = output.
- Pipeline = preprocess + model safely.
- cross_val_score = repeated evaluation.
- GridSearchCV = hyperparameter search.
- ColumnTransformer = different preprocessing per column type.

INTERVIEW FORMULA

- Definition -> why it matters -> one example -> limitation.
- If unsure, explain the workflow clearly instead of memorizing jargon.

Remember: strong SQL + clean data thinking + clear ML fundamentals are more valuable for entry-level analytics/data roles than memorizing dozens of algorithms.

Made By SK SAHIL

(Freelance AI Engineer)

Reference: Scikit-learn stable documentation (model selection, preprocessing, metrics and Pipeline APIs).